

Ideal performance benchmarking - Automation testing

Context

Performance benchmarking is done today on AA in a simpler way, while trying to mimic user sessions. Some of the aspects can be non-ideal or suboptimal, while targeting a linear simpler way of measurement to focus on Session/Test level optimisations.

(Current approach can be referred [here](#))

We wanted to revisit benchmarking to make it further closer to how actual user sessions run.

(cover Parallelism, Multi-tests etc)

Things to consider

1. Ensure all steps in execution are optimised and measured
 - a. Test session - Setup time
 - b. Outside session - Device waiting time, Parallelisation/Orchestration etc
2. Replicate customer's way of testing, and cover as many base as possible (above >50% if possible)
 - a. Framework, Features, Parallelism, Duration etc.
3. Slowness more likely to be perceived and felt for longer builds. Customer might complaint for a 30 min vs 50 min built but not for a 30 sec vs 50 sec build. **TO BE CONSIDERED**
 - a. Should ideal benchmarking focus more on longer builds?
 - b. How to filter out those?

Aspect wise Evaluation

✓ Logic of Build duration

Goal: Ensuring that 90% of the builds are performant for all groups.

- Any fixed setup time (e.g. Device setup, Device waiting, SDK) should be optimised and monitored for 90% of builds of a group.

Steps:

1. Calculate P10 build for each group (so remaining 90% of builds of that group would be higher than that duration). Data is [here](#).
(Measured for Q4 period)

2. Next step: Derive a single number representative of most groups, or safely solving for most groups. Options:
 - a. Mode (Representation of higher groups): ~50-75 sec bucket
 - b. Median - 147 seconds. Interpretation: (50% groups have (90% or more) builds >147 sec)
 - i. For overall number of builds (without group aggregation), this covers 92% of those.

Limitation: Above numbers too small, and not a good representation of longer builds for which the usual Customer issues are shared.

Proposal: Refinement - Choose builds only above a certain # of test runs (say with 1 or 2 or 3 tests which can be a proxy for smaller builds).

- Also ensure the majority builds (50%+) are not dropped.

Considering the mix of dropping shorter builds while maintaining a majority proportion, we consider builds with test runs ≥ 3 .

So final view = [Median aggregation of Group level P10 durations, while considering only Builds ≥ 3 test runs]

Detailed distribution of [Test runs here](#).

[Paid groups only]

	AA		Aut	
	Represents what % of Builds	Build duration (seconds)	Represents what % of Builds	Build duration (seconds)
All builds	100%	147	100%	108
Builds with Test Runs > 1	72%	216	86%	152

Builds with Test Runs > 2	64%	282	73%	186
Builds with Test Runs > 5	36%	514	55%	292

Open item: Check with CEs if this proxy is good enough for POC stage comparisons, or any feedback/refinements suggested?

▼ Free + Paid groups with Build data

	AA		Aut	
	Represents what % of Builds	Build duration	Represents what % of Builds	Build duration
All builds	100%	100	100%	79
Builds with Test Runs > 1	78%	140	87%	99
Builds with Test Runs > 2	65%	164	75%	119
Builds with Test Runs > 5	37%	356	51%	212

Sign in to access action bar for [Google Drive Spreadsheet](#)



Sign in to your Google Account

You must sign in to access this content

Sign in

Quick summary from evaluation

- Framework - Straightforward ✓
- Combination of Tests, Build and Parallels based on user data don't align directly (maths of `test duration * number of tests = parallels * build duration`) won't work with current data)
 - Testing duration - 800 seconds Build, 160 seconds test ?
 - Tests run - 3-5 tests ?
- Features - Current looks good for a base scenario. Individual feature level tests/sessions to be done.

Suggested approach

Have the base combination running because of:

- Ability to capture common buckets concretely and cleanly.
- Historical context and measurements for Local comparison, Competition comparison etc.

Further, **build an advanced combination** for better coverage of various scenarios. Use this for advanced assessments such as Feature level impact.

1. Create the user combination via multiple parallels (3), multiple tests (3).
 - a. Run on 3 parallels (setup local also similar).
 - b. Build duration (considering longer builds where Perf delta becomes more prominent) 282 seconds (for AA), 186 sec (for Aut).
2. Measure every time - inside session, outside session, all of (setup time, SDK time, waiting time, execution etc)
3. Different tests can cover base + advanced scenarios (LocalOn, Network Logs, Top Device features etc), and if measured right, be used for respective features.

Numbers will be **reported for both base and advanced**, and we should target proximity to Local and better than Competition in both scenarios.

- Over time, if we have gained confidence on one of this (mostly Advanced comb.) to cover Performance in the optimal way, we can discard the other.

Benchmarking format (slightly unrelated)

This is irrespective of **Base or Advanced** benchmarking.

Benchmarking format:

The current benchmarking reports are split across regions and currently being sent in mails which are difficult to keep track off in the long term. The requirements for the ideal format of performance benchmarking is given below:

1. Consider the P90 metrics to show as a part of the email reporting. By default the P90 metrics are shown. However aggregated P50 metrics would also be included in the report. For region wise P50 report - the dashboards will have to be referred to
2. Current grouping is on a Region > platform level given certain regions have longer expected start times due to Dc/hub/repeater allocation
3. Collate all the metrics across competitors and regions in a single table within the email being sent. Example data for 2 regions as below

Overall metrics (All regions)					
Platform	P90 session duration	P50 session duration			
Android	235.53	178.14			
iOS	235.53	178.14			

Platform P90 metrics						
Platform	Metric	Browserstack	LT	Sauce	ack vs Lambda	stack vs Sauce (%)
Android	Start time	19.11	26.36	15.41	#ERROR!	-19.36% Slower
	Execution time	178.14	235.53	221.15	#ERROR!	#ERROR!
	Stop time	15.44	32.28	47.48	#ERROR!	#ERROR!
iOS	Start time	19.11	26.36	15.41	#ERROR!	#ERROR!
	Execution time	178.14	235.53	221.15	#ERROR!	#ERROR!
	Stop time	15.44	32.28	47.48	#ERROR!	#ERROR!
Platform P50 metrics						
Platform	Metric	Browserstack	LT	Sauce	ack vs Lambda	stack vs Sauce (%)
Android	Start time	19.11	26.36	15.41	#ERROR!	-19.36% Slower
	Execution time	178.14	235.53	221.15	#ERROR!	#ERROR!
	Stop time	15.44	32.28	47.48	#ERROR!	#ERROR!
iOS	Start time	19.11	26.36	15.41	#ERROR!	#ERROR!
	Execution time	178.14	235.53	221.15	#ERROR!	#ERROR!
	Stop time	15.44	32.28	47.48	#ERROR!	#ERROR!
P90 Execution Time Comparison						
Region	Platform	Browserstack	Lambda	Sauce	ack vs Lambda	stack vs Sauce (%)
USE	android	123.99	161.67	134.23	#ERROR!	#ERROR!
	ios	178.14	235.53	221.15	#ERROR!	#ERROR!
APS	android	123.99	161.67	134.23	#ERROR!	#ERROR!
	ios	178.14	235.53	221.15	#ERROR!	#ERROR!
P90 Start time Comparison						
Region	Platform	Browserstack	Lambda	Sauce	ack vs Lambda	stack vs Sauce (%)
USE	android	19.11	26.36	15.41	#ERROR!	-19.36% Slower
	ios	15.44	32.28	47.48	#ERROR!	#ERROR!
APS	android	123.99	161.67	134.23	#ERROR!	#ERROR!
	ios	178.14	235.53	221.15	#ERROR!	#ERROR!
P90 Stop time Comparison						
Region	Platform	Browserstack	Lambda	Sauce	ack vs Lambda	stack vs Sauce (%)
USE	android	19.11	26.36	15.41	#ERROR!	-19.36% Slower
	ios	15.44	32.28	47.48	#ERROR!	#ERROR!
APS	android	123.99	161.67	134.23	#ERROR!	#ERROR!
	ios	178.14	235.53	221.15	#ERROR!	#ERROR!

- HyperExecute metrics along with Bstack vs HyperExecute needs to be added to the above email.
- The subject line of the above mail should have the specific capability base info eg video off video on etc. Would be better if this could be customised by the user
- Expose the same on admin dashboard so that PM's can trigger the respective on demand if required
- All the benchmark data needs to be added to a new BQ table with the above avg metrics for start time, session duration, stop time, number of sessions, region, competitor metrics, date, timestamp of test, capabilities and the respective hash id's/build id's.
- Both P90, P50 metrics need to be captured for every run and added to the database for future reference